

THE DATA

I did not type my rows by hand. A Python loader walks through my Obsidian vault and renders it as the multi-row INSERT statements in the sql file. The thing I thought hardest about was NULL. My HW9 design had identity_tag_id, maturity_stage_id, date_created and date_modified all NOT NULL, on the reasoning that every note obviously has an author intent, a maturity and a creation date. However, in practice, not every note's file says so. I could have defaulted those to something plausible, and I nearly did, because a timestamp of now() would have made the load run without complaint. However, I did think that, the difference between “this note was created in August 2026” and “I do not know when this note was created” is exactly the difference NULL exists to express, so I made those four columns nullable and let the gaps show, also with the hopeful intent that I might go back and fill those with a better guesstimate of the date rather than whatever now() time of the data being transferred from markdown to the database.

A QUERY I AM PROUD OF

Q8 asks what the three healthiest notes are under each substantial topic:

```
WITH ranked AS (  
    SELECT  
        v.topic,  
        v.note_count AS topic_note_count,  
        n.title,  
        fn_note_health(n.node_id) AS health_score,  
        RANK() OVER (  
            PARTITION BY v.topic_tag_id  
            ORDER BY fn_note_health(n.node_id) DESC  
        ) AS rank_in_topic  
    FROM v_topic_activity v  
    JOIN node_topics nt ON nt.topic_tag_id = v.topic_tag_id  
    JOIN nodes n ON n.node_id = nt.node_id  
    WHERE v.note_count >= 10  
)  
SELECT topic, topic_note_count, rank_in_topic, title, health_score  
FROM ranked  
WHERE rank_in_topic <= 3  
ORDER BY topic, rank_in_topic, title;
```

The shape is a window function wrapped in a CTE, and the CTE is there for a specific reason rather than for decoration: a window function cannot appear in a WHERE clause, because ranking happens after filtering, so the rank has to be computed in one query and filtered in the

next. The PARTITION BY is what makes it a per-topic answer instead of a vault-wide one; without it, Data Science and Machine Learning would fill every slot and the smaller topics would never appear. It reads from my view, which supplies the topic-level totals and lets me exclude topics with fewer than ten notes, since a top three out of three is not a ranking. It also calls my function, `fn_note_health`, which scores a note on its sources, its incoming and outgoing links and its aliases, and then divides that score by how long the note has sat untouched.

What surprised me was the literature notes gap from the technical note. I did expect the technical topics to win, and they did, the Clustering note scores 23.95, but I did not expect the gap. The best note under Vietnamese Literature scores 3.96 and the best under Mathematics scores 5.42, while Psychology's best scores 17.82.

SERVER-SIDE LOGIC

My trigger writes to `node_stage_audit` every time a note's maturity stage changes, recording where it moved from, where it moved to, and whether that was a promotion or a demotion.

It belongs in the database because the database has more than one writer. Rows arrive through my sync command, but I also run UPDATE statements by hand in `psql` while testing, and a future version of this project will almost certainly have a second path in. A trigger is the only place where the rule “every stage change gets recorded” is true regardless of who is doing the changing. There is also a correctness argument that has nothing to do with discipline: the trigger's INSERT happens inside the same transaction as the UPDATE that fired it, so the audit row and the change it describes commit together or not at all. My T2 rollback demonstrates that the audit row vanishes along with the promotion.

If I were writing a Python or Java application against this database, this logic would have gone in a service method, something like `NoteService.promote()`, which would open a transaction, update the note and insert the audit row before committing. I would have gained real things.

The logic would be readable by anyone who knows the application language rather than only by someone who knows `pgSQL`. It would be unit-testable without a running Postgres; it would live in version control alongside the code it serves instead of inside a schema migration; and it could do things SQL cannot, like notifying me that a note had matured. What I would have lost is the guarantee. Every write path would have to remember to go through that method, and the moment I opened `psql` to fix something by hand, the audit trail would silently stop being complete, and a silently incomplete audit trail is worse than none, because I would still trust it.

THE ROLLBACK

The rule that forced my rollback is this: a note may not be promoted to adult until it cites at least one source and links out to at least two other notes. Adult is supposed to mean finished, and a finished note in a Zettelkasten is one that is grounded in something I read and connected to something I already knew. In T2 I promote MasteryCalculation to adult and add one link to Mastery Learning in the same transaction, then read my own uncommitted work and find one outgoing link and zero sources. The rule is broken, so I roll the whole thing back, and both the promotion and the link disappear.

A CHECK constraint could never have enforced this. A CHECK is evaluated against a single row of a single table, using only that row's own column values, and this rule is a statement about how many rows exist in two other tables. The count of a note's links lives in node_links and the count of its sources lives in node_sources, and nodes has no column that knows either number.

Postgres will not even let me write the subquery that would be required, and that restriction is not an arbitrary limit, it is protecting me from something unsound. Even if the subquery were allowed, the CHECK would only fire when the nodes row itself was written. Deleting a link afterwards would leave an adult note with one link and no constraint would ever look again. The rule has to be evaluated after the writes and across tables, which means it belongs where I put it: inside the transaction, after the work, with a rollback waiting if the answer comes back wrong.